PHASE 2

Architecture and Environment Configuration

Request pipeline, folder structure and environment variables

COMICCRAFT • AI COMIC STORY CREATOR • TECHNICAL DOCUMENTATION

1. Application Architecture

The request lifecycle follows a five-stage pipeline orchestrated by comic_service.create_comic():

1. Outline — Gemini Flash converts the prompt into a structured panel-by-panel outline (title, scene description, image prompt per panel).
2. Story — Gemini Pro expands the outline into narration, captions and dialogue for every panel.
3. Illustration — each panel's image prompt is rendered to a PNG by the configured image provider.
4. Layout — outline, story and image results are merged into renderable panel objects.
5. Export — the finished layout is optionally rendered to a print-ready PDF via FPDF2.

1.1 Folder Structure

```
ComicCraft/
├── app/
│   ├── main.py                # App factory, static mount, global error handling
│   ├── routes.py               # All HTTP routes
│   ├── schemas.py              # Pydantic models
│   ├── config.py                # Env-driven settings
│   ├── templating.py             # Shared Jinja2 environment
│   ├── services/
│   │   ├── gemini_client.py        # Shared Gemini SDK wrapper
│   │   ├── gemini_flash.py          # generate_outline()
│   │   ├── gemini_pro.py             # generate_story()
│   │   ├── image_generator.py         # Provider abstraction + generate_image()
│   │   ├── layout_builder.py           # build_comic_layout(), generate_panel_images()
│   │   ├── exporters.py                 # save_pdf()
│   │   └── comic_service.py              # Pipeline orchestration
│   └── utils/
│       ├── validators.py       # AI output normalisation
│       └── helpers.py           # Safe paths, filenames, JSON extraction
├── templates/                    # base.html, index.html, comic_preview.html, ...
├── static/
│   ├── css/style.css
│   ├── js/app.js
│   ├── panels/                 # Generated panel images
│   └── exports/                # Generated PDFs
├── tests/
├── .env.example
├── requirements.txt
└── run.py
```

2. Environment Configuration

All secrets and model identifiers are read from environment variables (via a .env file, loaded with python-dotenv). Copy .env.example to .env before first run.

Variable	Purpose
GEMINI_API_KEY	Required. Google AI Studio key used for both text models.
GEMINI_OUTLINE_MODEL	Model used for the structured panel outline.

Variable	Purpose
GEMINI_STORY_MODEL	Model used for narration and dialogue.
IMAGE_PROVIDER	stable_diffusion, huggingface_api, gemini_image, or placeholder.
IMAGE_MODEL	Diffusion model id, e.g. runwayml/stable-diffusion-v1-5.
HF_API_KEY	Hugging Face token — required for huggingface_api.
PANEL_COUNT	Panels per comic (default 5).
MAX_PROMPT_LENGTH	Server-side prompt limit, mirrored by the UI counter.
DEBUG / APP_ENV	Development toggles.

Security note
Keys are read server-side only and are never sent to the browser. .env is gitignored. IMAGE_PROVIDER=placeholder is an explicit opt-in offline mode for UI work — it is never selected automatically.